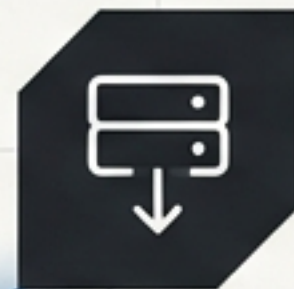
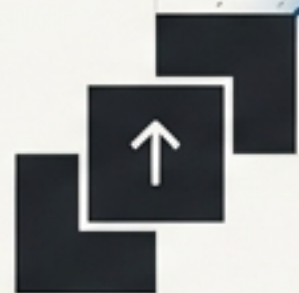


数字世界的握手

深入解析 SSH 连接的每个环节



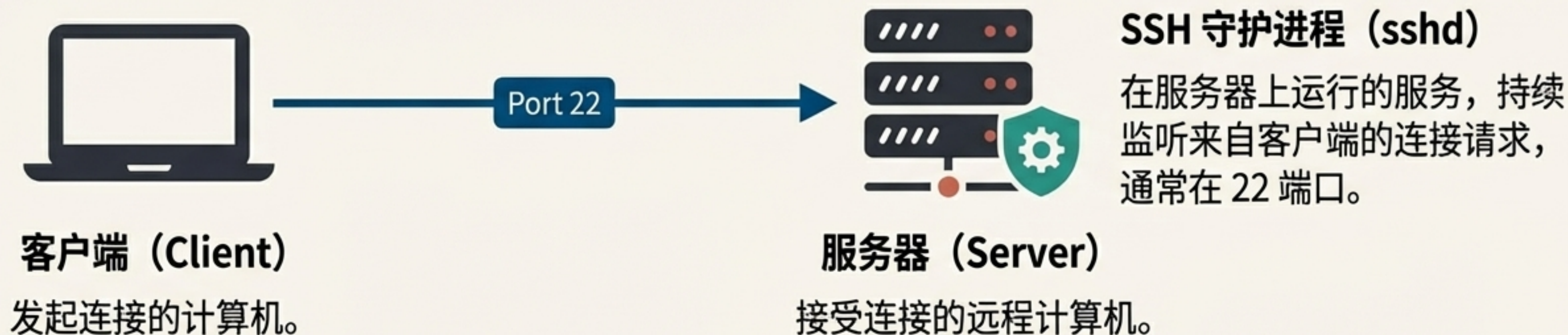
在不安全的网络中，信任从何而来？



我们的数字通信建立在开放且默认不受信任的网络之上。如果没有安全措施，任何数据传输都可能面临两大核心威胁：

- **窃听** (Eavesdropping)：第三方可以读取您的敏感数据，如密码和私密信息。
- **中间人攻击** (Man-in-the-Middle Attack)：攻击者可以冒充通信的另一方，拦截、读取甚至篡改您的数据。

SSH 的基本架构：连接的参与者



一次安全握手的六个步骤

建立一个安全的 SSH 会话需要经过一个严谨、有序的六阶段过程。每个阶段都为最终的安全通信通道贡献关键的一环。



阶段 1 & 2：初步接触与规则同步

阶段 1：建立 TCP 连接 (Phase 1: TCP Connection)

客户端向服务器的 22 端口（或自定义端口）发起一个标准的 TCP 连接。通过三次握手，一个一个可靠的传输层通道被建立。



阶段 2：协议版本交换 (Phase 2: Protocol Version Exchange)

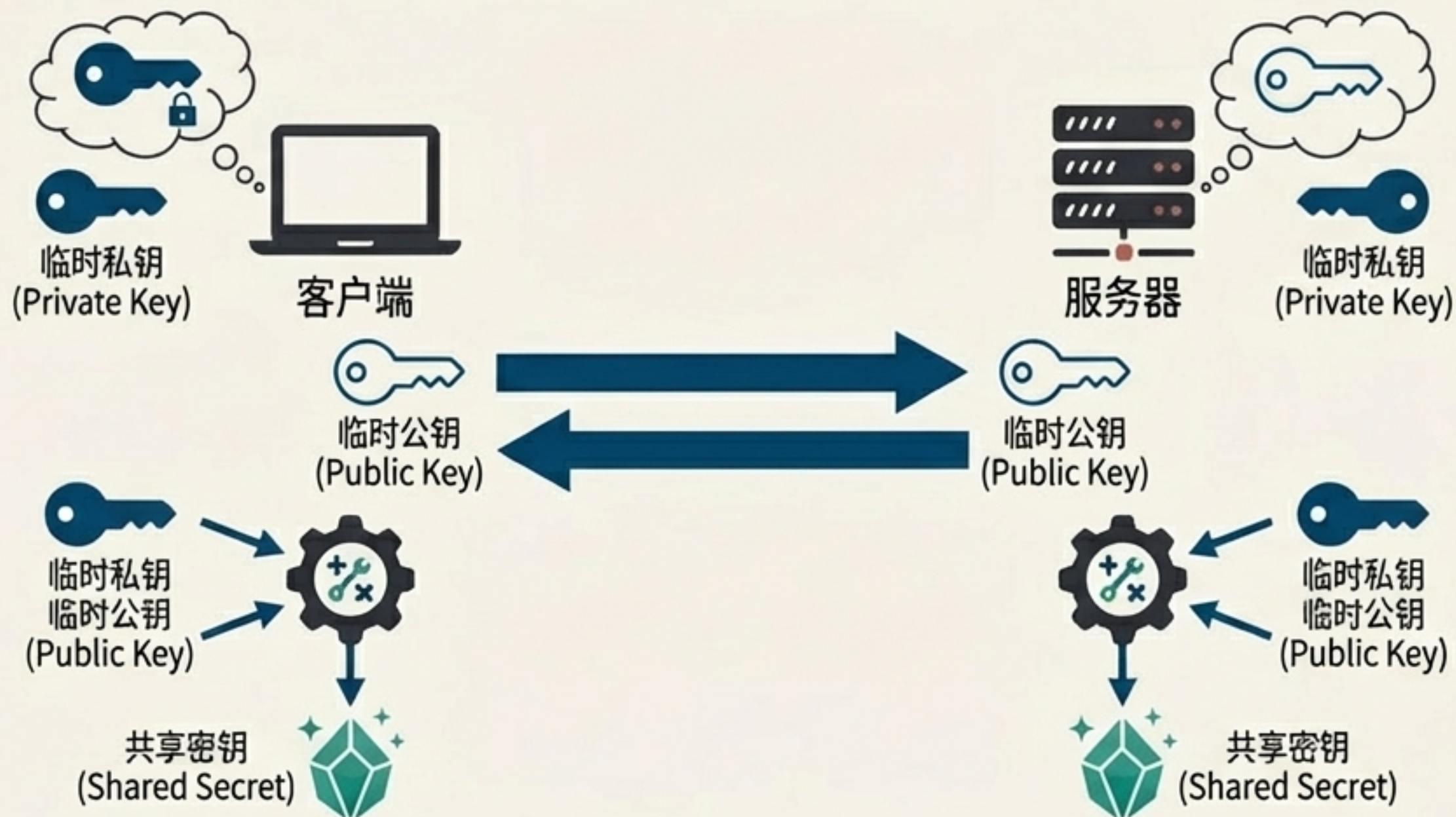
客户端和服务端各自发送自己的 SSH 协议版本号（例如，SSH-2.0）。双方确认使用兼容的协议版本，为后续的加密协商设定基础。

一次安全握手的六个步骤

建立一个安全的 SSH 会话需要经历一个严格、双向的认证过程。每个步骤都为最终的安全通信奠定基础。

1. TCP 连接 (TCP Connection)
2. 协议版本交换 (Protocol Version Exchange)
3. 密钥交换 (Key Exchange)
4. 服务器认证 (Server Authentication)
5. 客户端认证 (Client Authentication)
6. 会话建立 (Session Establishment)

阶段 3：密钥交换 - 在公开场合创建一个秘密



一次安全握手的六个步骤

建立一个安全的 SSH 会话需要经过一个过程，其中 SSH 协议版本、算法和密钥交换算法的安全级别都会被协商确定。

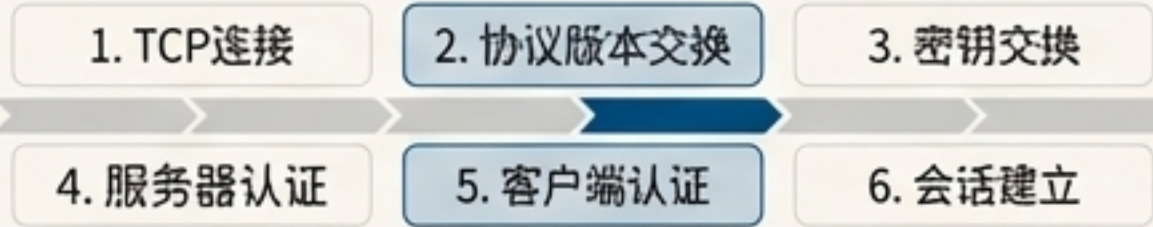
1. TCP 连接 (TCP Connection)
2. 协议版本交换 (Protocol Version Exchange)
3. 密钥交换 (Key Exchange)
4. 服务器认证 (Server Authentication)
5. 客户端认证 (Client Authentication)
6. 会话建立 (Session Establishment)

1. 客户端和服务端各自生成一个临时的、一次性的密钥对（公钥+私钥）。
2. 它们互相交换各自的临时公钥。
3. 双方使用自己的临时私钥和对方的临时公钥，通过 Diffie-Hellman 算法各自独立计算出完全相同的共享密钥 (Shared Secret)。



Security Benefit: 这个共享密钥从未在网络上传输过，窃听者即使截获了所有交换的数据（两个临时公钥），也无法计算出它。这个密钥是后续所有加密的基础。

阶段 4：服务器认证 - “你是你所声称的那个服务器吗？”



Security Benefit: 这一步至关重要，用于防止中间人攻击。如果收到的主机密钥与已存储的不匹配，SSH 客户端会发出严重警告，提示连接可能被劫持。

深入了解：SSH 身份的基石 - 密钥



主机密钥 (Host Keys) - 服务器的身份

用途：长期用于向客户端证明服务器的身份。

存储位置 (服务器端)：

/etc/ssh/ssh_host*_key (私钥) 和 /etc/ssh/ssh_host*_key.pub (公钥)。

常见类型： `RSA`，ECDSA，`Ed25519`。



已知主机 (Known Hosts) - 客户端的信任记录

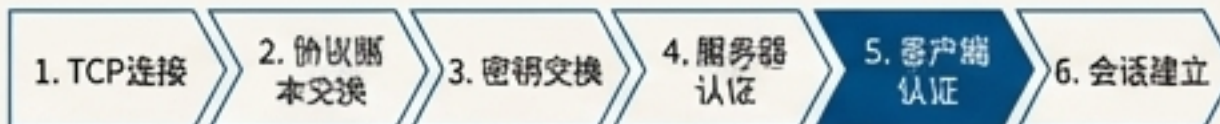
用途：客户端存储所有它信任的服务器的主机公钥。

存储位置 (客户端)：

~/.ssh/known_hosts。

机制：每次连接时用于验证服务器身份，防止中间人攻击。

阶段 5：客户端认证 - “你是否有权访问？”



服务器验证客户端的身份，以确保只有授权用户可以访问。SSH 支持多种方法，最常见的是：

a) 密码认证 (Password Authentication)

过程：客户端将用户输入的密码加密后发送给服务器。服务器解密并与系统存储的凭证进行验证。

缺点：容易受到暴力破解攻击，且密码可能在客户端被窃取。不推荐在生产环境中使用。

b) 公钥认证 (Public Key Authentication) - 推荐

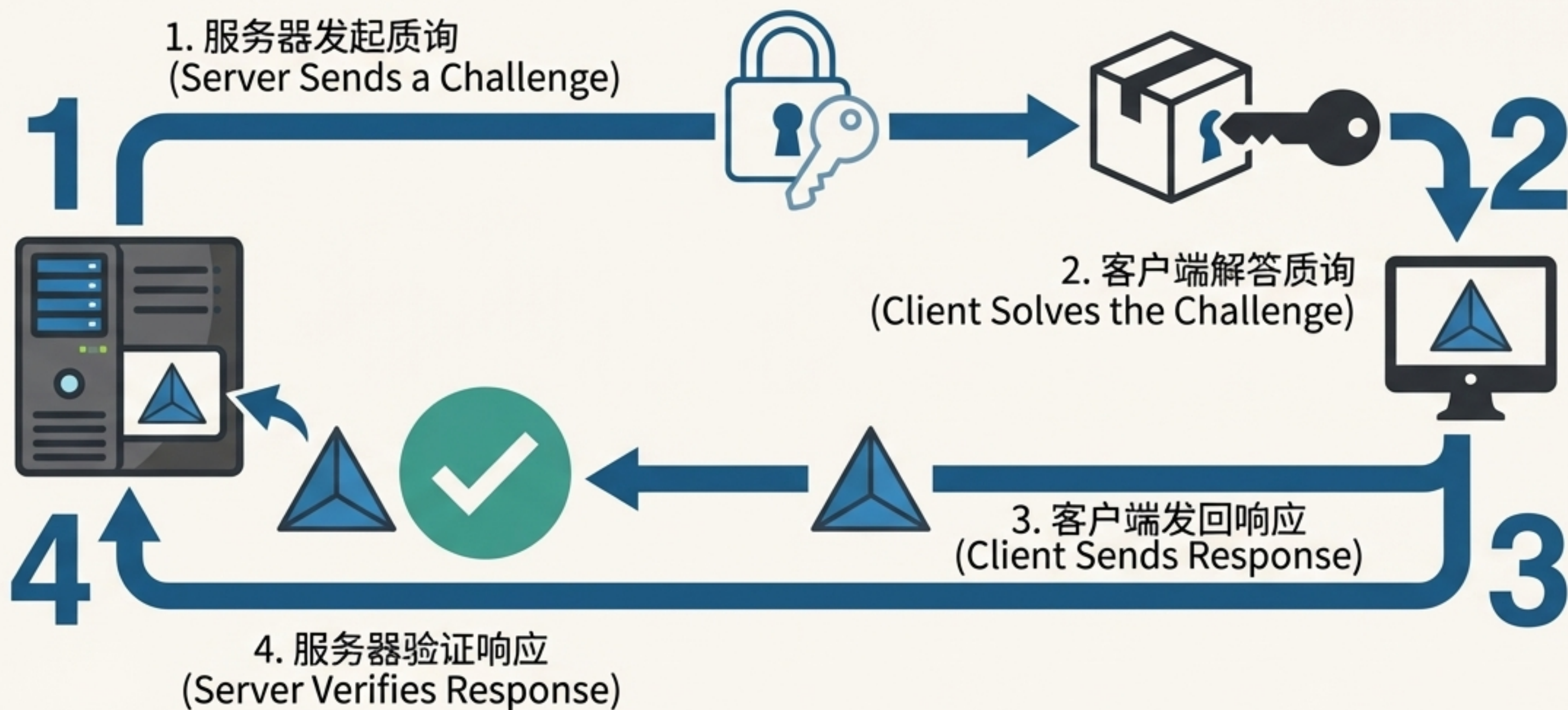


过程：一种更安全、基于密码学的质询-应答机制。

优点：私钥不出客户端，无法被暴力破解，可实现无密码登录。

深度解析：公钥认证的工作原理

Prerequisite: 客户端的公钥必须预先存放在服务器的 `~/.ssh/authorized\_keys` 文件中。



深入了解：用户密钥 - 你的个人数字身份



用户密钥对 (User Key Pair) - 客户端的凭证

私钥 (Private Key)



KEEP SECRET

用途：你的个人秘密，用于向服务器证明你的身份。**绝不能泄露。**

存储位置：`~/ .ssh/id\_ed25519`, `~/ .ssh/id\_rsa`, etc.

公钥 (Public Key)



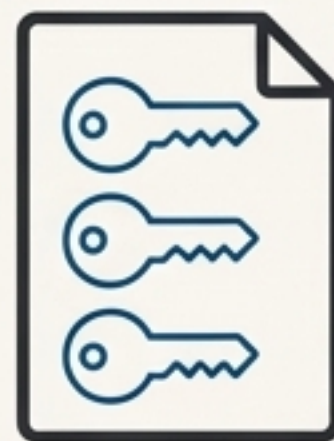
SHARE FREELY

用途：可以安全地分发给任何人或系统，用于验证你的身份。

存储位置：`~/ .ssh/id\_ed25519.pub`,
`~/ .ssh/id\_rsa.pub`, etc.



授权密钥 (Authorized Keys) - 服务器的访问列表



用途：一个纯文本文件，包含了所有被授权访问该服务器账户的用户的**公钥**。

存储位置 (服务器端)：`~/ .ssh/authorized\_keys`.

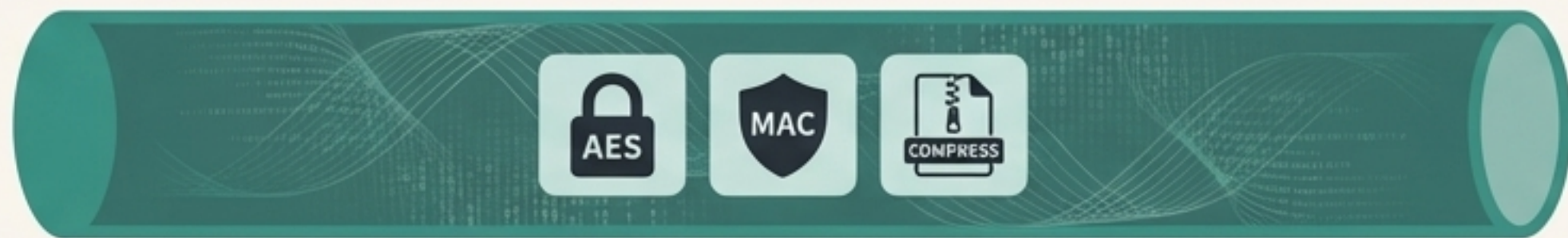


阶段 6：会话建立 - 协商通信规则



认证成功后，双方将使用在阶段 3 生成的共享密钥来派生出一系列会话密钥（Session Keys）。然后，它们协商并同意本次会话使用的具体算法，包括：

- **加密算法（Encryption Algorithms）**：用于保护数据机密性。例如：`AES-256`，`ChaCha20-Poly1305`。
- **消息认证码算法（MAC Algorithms）**：用于确保数据完整性，防止篡改。
- **压缩算法（Compression Algorithms）**：用于在传输前压缩数据，提高效率。



一旦协商完成，一个完全加密和受保护的 Shell 通道或数据连接就正式建立了。

成果：一个坚不可摧的安全通道

成功的 SSH 连接提供了多层安全保障：



加密 (Encryption)：
所有传输的数据（包括密码、命令、文件内容）都使用强大的对称加密算法进行加密。会话密钥由初始密钥交换派生而来。

完整性 (Integrity)：
每个数据包都包含一个消息认证码 (MAC)，确保数据在传输过程中没有被篡改。

认证 (Authentication)：
服务器认证防止中间人攻击，客户端认证确保只有授权用户可以登录。

前向保密 (Forward Secrecy)： 每个 SSH 会话都使用独特的、临时的会话密钥。即使一个会话的密钥被泄露，也不会影响过去或未来的任何其他会话的安全。

掌握这门艺术：SSH 安全最佳实践



始终使用公钥认证：禁用密码认证。



使用更强的密钥算法：优先选择 Ed25519 而不是 RSA。



为私钥设置强密码（Passphrase）：为你的私钥增加一道额外的安全屏障。



定期更新 SSH 软件：及时修复已知的安全漏洞。



禁用 root 用户远程登录：使用普通用户登录，必要时再提权。



使用 Fail2ban：自动阻止尝试暴力破解的 IP 地址。



定期轮换密钥：遵循组织的安全策略。

从开始到结束：SSH 连接全流程回顾

